

自觉遵守考场纪律
如考试作弊
此答卷无效

姓名
封
密
学号

东南大学 考试卷（A 卷）

课 程 名 称	数据结构	考 试 学 期	08-09-3	得 分	
适 用 专 业	吴健雄学院电类	考 试 形 式	半开卷	考试时间长度	120 分钟

一、选择题（每题 1 分，共 5 分）

1. 下面有关链栈的描述，对常规情况正确的是（ ）
A. 在链头插入，链尾删除。 B. 在链尾插入，链头删除。
C. 在链尾插入，链尾删除。 D. 在链头插入，链头删除。
2. 对线性表进行对半搜索时，要求线性表必须（ ）
A. 以数组方式存储 B. 以数组方式存储并按关键码排序
C. 以链表方式存储 D. 以链表方式存储并按关键码排序
3. 对包含 n 个元素的散列表进行搜索，平均搜索长度为（ ）
A. $O(\log_2 n)$ B. $O(n)$ C. 不直接依赖于 n D. 三者均不是
4. 在同一个有向图中，所有结点的入度和与出度和之比为（ ）
A. 1 B. 2 C. 1/2 D. 都不对
5. 在具有 n 个顶点的无向图中，要连通全部顶点至少需要（ ）条边。
A. n B. $n+1$ C. $n-1$ D. $n/2$

二、判断题（每题 1 分，共 5 分）

1. 链式存储的线性表所有存储单元的地址可连续可不连续。 ()
2. 存储有向图的邻接矩阵是对称的，所以可以仅存矩阵上三角部分。 ()
3. 在采用闭散列法解决冲突时，不要立刻做物理删除，否则搜索时会出错。 ()
4. 二叉树中序遍历结果序列的最后一个结点必是前序遍历的最后一个结点。 ()
5. 堆排序的时间复杂度是 $O(n \log_2 n)$ ，但需要额外存储空间。 ()

三、填空题（每空 1 分，第 1 空、第 2 空为 2 分，共 11 分）

1. 中缀表达式 “ $(a+b)*d+e/(f+a*d)+c$ ” 所对应的后缀表达式为
_____(1)_____
2. 后缀表达式 “ $ab\&\&ef>!\|$ ” 所对应的中缀表达式为 (____2)_____

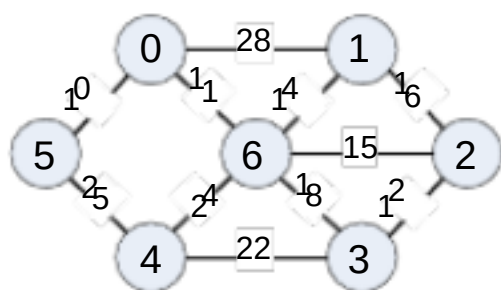
3. 高度为 h 的二叉树最多可以有多少结点 (3)
4. 若对一棵完全二叉树从 0 开始编号, 并按此编号把它顺序存储到一维数组 a 中, 则 $a[i]$ 元素的左孩子结点为 (4), 右孩子结点为 (5), 双亲结点为 (6)。
5. 对用邻接矩阵表示的图进行任何一种遍历时, 其时间复杂度为 (7)。对用邻接表表示的图进行任何一种遍历时, 其时间复杂度为 (8)。
6. 折半插入排序的时间复杂度为 (9)。

四、简答简述题 (每题 8 分, 共 24 分)

1. 设有一组关键码输入序列 {55, 31, 12, 37, 46, 73, 63, 02, 07}, 从空树开始构造平衡二叉搜索树, 画出每加入一个新结点时的二叉树形态, 需标出平衡因子。包括发生不平衡时, 旋转的各步的二叉树形态, 并标注旋转类型。

2. 已知一棵二叉树的前序遍历的结果为 ABECDFGHIJ, 中序遍历的结果是 EBCDAFHIGJ, 试画出这棵二叉树。请用图表示逐步形成二叉树的过程 (也可以用文字)。

3. 请用 Kruskal 算法，逐步画出下面有权无向图的最小生成树。必须每次添加一条边。



五、综合算法题（每空 2.5 分，共 55 分）

1. 完善改进的归并排序算法。 *this 是一个待排序的表，而表 L2 是一个辅助的工作表，帮助完成排序的中间步骤，最终完成 *this 的排序。所谓改进指在把元素序列复制到辅助表中时，把第 2 个表的元素顺序逆转，这样两个待归并的表从两端开始处理，向中间归并。可以省去检查子表是否结束的判断。

```
template <typename T>void Orderedlist<T>::MergeSort(int left, int right){
    Orderedlist<T> L2;
    improvedMergeSort(L2, left, right);          //对序列进行归并排序
}
template <typename T>
void Orderedlist<T>::improvedMergeSort(Orderedlist<T> &L2, int left, int right){
    int mid = (left + right)/2;                  //从中间划分为两个子序列
    improvedMergeSort(L2, left, mid);            //对左侧子序列进行归并排序
    improvedMergeSort(L2, mid + 1, right);        //对右侧子序列进行归并排序

    (1)_____;//二路归并
}
template <typename T>
void Orderedlist<T>::improvedMerge(Orderedlist<T> &L2, int left, int mid, int right){
    int s1 = left, s2 = right, t = left, k ;      //s1,s2是检测指针， t是存放指针
    for (k = left; k <= mid; k++){                //正向复制

        (2)_____;
    }
    for (k = mid + 1; k <= right; k++){           //反向复制

        (3)_____;
    }
    while (t <= right){                            //归并过程

        if(L2.slist[s1] <= L2.slist[s2]) (4)_____;

        else (5)_____;
    }
}
```

2. 完成二叉树前序遍历的非递归算法和层次序遍历算法操作。

//非递归前序遍历。每访问一个结点后，在向左子树遍历下去之前，利用栈记录该结点的右子女结点的地址，以便在左子树退回时可以直接从栈顶取得右子树的根结点，继续右子树的前序遍历。


```

template <typename T>void BinaryTree<T>::PreOrder1(void (*visit) (BinTreeNode<T> *t) ) {
    LinkedStack<BinTreeNode<T>*> S;
    BinTreeNode<T> *p = root;
    S.Push (NULL);
    while (p != NULL) {
        visit(p);                //访问结点
        if (p->rightChild != NULL)
            (6)_____; //预留右指针在栈中
        if (p->leftChild != NULL)
            (7)_____; //进左子树
        else (8)_____; //左子树为空，由堆栈弹出
    }
}

```

//层次序遍历。在访问二叉树某一层结点时，把下一层结点指针预先记忆在队列中，利用队列安排逐层访问的次序。因此每当访问一个结点时，将它的子女依次加到队尾。然后访问已在队头的结点。

```

template <typename T>void BinaryTree<T>::levelOrder (void (*visit) (BinTreeNode<T> *t)) {
    if (root == NULL) return;
    LinkedQueue<BinTreeNode<T> * > Q;
    BinTreeNode<T> *p = root;
    visit (p);
    (9)_____ ;
    while ( (10)_____ ) {
        Q.DeQueue (p);
        if (p->leftChild != NULL) {
            visit (p->leftChild);
            (11)_____ ;
        }
        if (p->rightChild != NULL) {
            visit (p->rightChild);
            (12)_____ ;
        }
    }
}
}

```

3. 完成利用最大堆实现的优先级队列类定义。注意 heap[0]不用，从 heap[1]开始

```

template<typename T>class Maxheap{
    Element<T>* heap;
    int n;
    int MaxSize;
public:
    Maxheap(int sz=Defaultsize); // 创建空堆，最多可以容纳    sz个元素
    void Insert(Element<T>& item);
    Element<T>* Delete(Element<T>& x);
    void show() ;
};

template<typename T>Maxheap<T>::Maxheap(int sz){
    MaxSize=sz;
    n=0;
    heap= new Element<T>[MaxSize+1]; // 注意 heap[0] 不用，从 heap[1] 开始
}

template<typename T>void Maxheap<T>::Insert(Element<T>& x){
    int i;
    if(n==MaxSize){
        cerr<<"heap is full.\n";
        return;
    }
    n++;
    for(i=n;i>1;){
        //i==1 表示已达到根节点
        if( (13)_____ ) break; // 新元素不大于结点 i 的双亲，不处理

        (14)_____;//此时 heap[i] 未占用，将双亲结点元素移入

        (15)_____;          //i继续向上
    }
    heap[i]=x;                //位置定了数值再放进去
}

template<typename T>Element<T>* Maxheap<T>::Delete(Element<T>& x){
    int i,j;
    if(!n){
        cerr<<"heap is empty.\n";
        return NULL;
    }
    x=heap[1];
    Element<T> k=heap[n];
    n--;

```

```

for(i=1,j=2;j<=n;){
    //j是i的子女
    if(j<n) if( (16) ) j++; //j 指向较大子女
    if(heap[j]<=k) break; //候补的结点大，不再移动
    (17); //还需移动， 将较大子女直接移入
    (18); //移动
    (19);
}
heap[i]=k;
return &x;
}

```

4. 完成的深度优先遍历图算法。

//深度优先遍历图，输出所有的连通分量

```

template<typename T, typename E> void Graph<T,E>::DFS(){
    int i, n = NumberOfVertices(); //取图中顶点个数
    bool *visited = new bool[n]; //创建辅助数组
    for (i = 0; i < n; i++){ //辅助数组 visited 初始化
        visited[i] = false;
    }
    for(i=0;i<n;i++){ //从每个顶点开始，各做一次遍历。
        if( (20) ){ //借助辅助数组，上一趟遍历已访问过的
            //各顶点不会作为新起点。所以输出了所有连通分量，不会重复。
            DFS(i, visited); //从顶点 0开始深度优先搜索
            cout<<endl;
        }
    }
    delete [] visited; //释放 visited
}

```

//从顶点位置 v出发，以深度优先的次序访问所有可读入的尚未访问过的顶点。

//算法中用到一个辅助数组 visited，对已访问过的顶点作访问标记。

```

template<typename T, typename E>void Graph<T,E>::DFS(int v, bool visited[]){
    cout << getValue(v) << ' '; //访问顶点 v
    visited[v] = true; //顶点 v作访问标记
    int w = getFirstNeighbor(v); //找顶点 v的第一个邻接顶点 w
    while (w != -1){ //若邻接顶点 w存在。注意邻接顶点数目不定
        if ( (21) ){ //若w未访问过，递归访问顶点 w

```

```
        (22) _____;  
    }  
    w = getNextNeighbor(v, w);    //取 v排在 w后的下一个邻接顶点  
}  
}
```